

final lab project - VHDL coding

Yuval Elron / 318676061
Amit Postelnik / 206458333

Project Goal

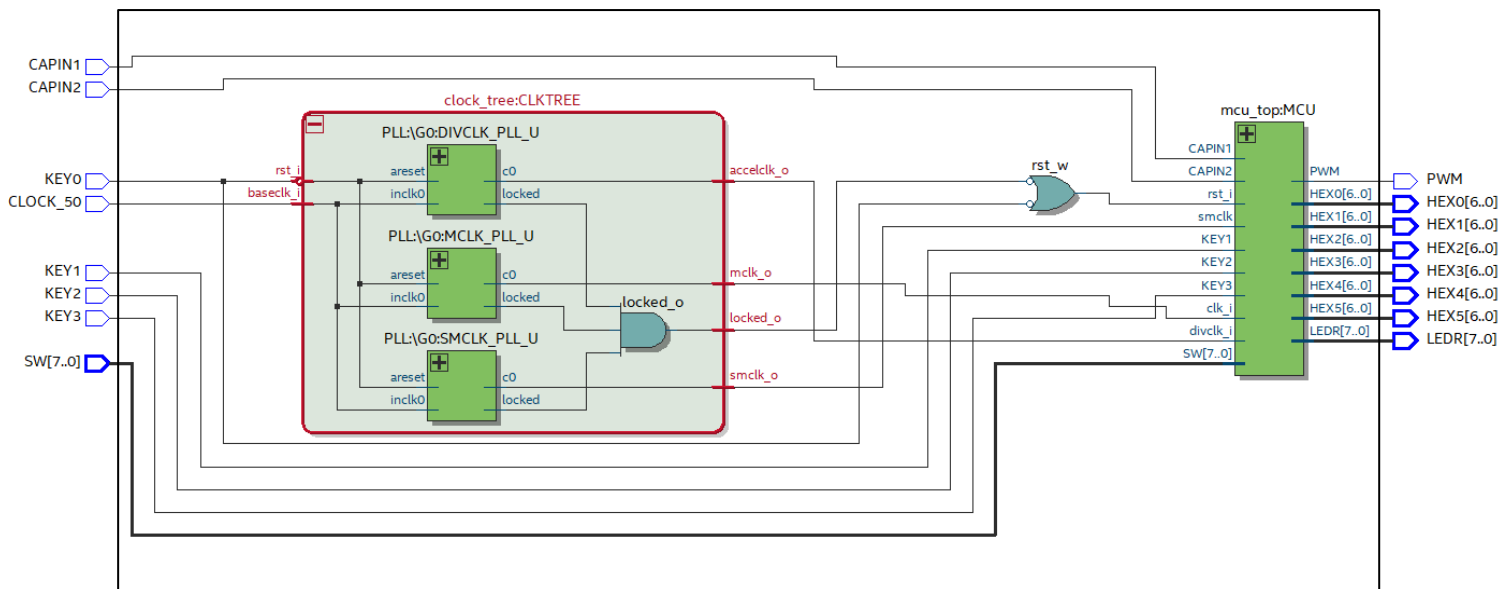
The goal of this project was to design, code and verify in VHDL a complete single-cycle RV32IM microcontroller system on an FPGA, and to extend it from a bare processor core into a working MCU with real memory-mapped peripherals and a hardware interrupt mechanism.

Table of Contents

Pages

Project Goal	1
part 1 – Top-level block diagram review	3
1.1) MCU_TOP	4
1.1.1) bidir_pin	5
1.1.2) basic_timer_top	7
1.1.2.1) addr_decoder_basic_timer	8
1.1.2.2) basic_timer	9
1.1.2.2.1) output_unit	10
1.1.2.2.2) bit_timer	11
1.1.3) pushbutton_peripheral	12
1.1.4) interrupt_controller_top	13
1.1.4.1) addr_decoder_interrupt	14
1.1.4.2) interrupt_controller	15
1.1.5) gpio_peripherals	17
1.1.6) RV32-CORE	19
1.1.7) sync and divider_accelerator	21
2) PPA Characterization of RV32IM-Based MCU Design	23
2.1) Supporting Quartus Reports – with interrupts	24
2.2) MCU with GPIO only (no interrupts)	27
3) SignalTap use	30

part 1 : we will review the Top-level block review diagram of Ur design



Picture 1 : MUC_SYSTEM

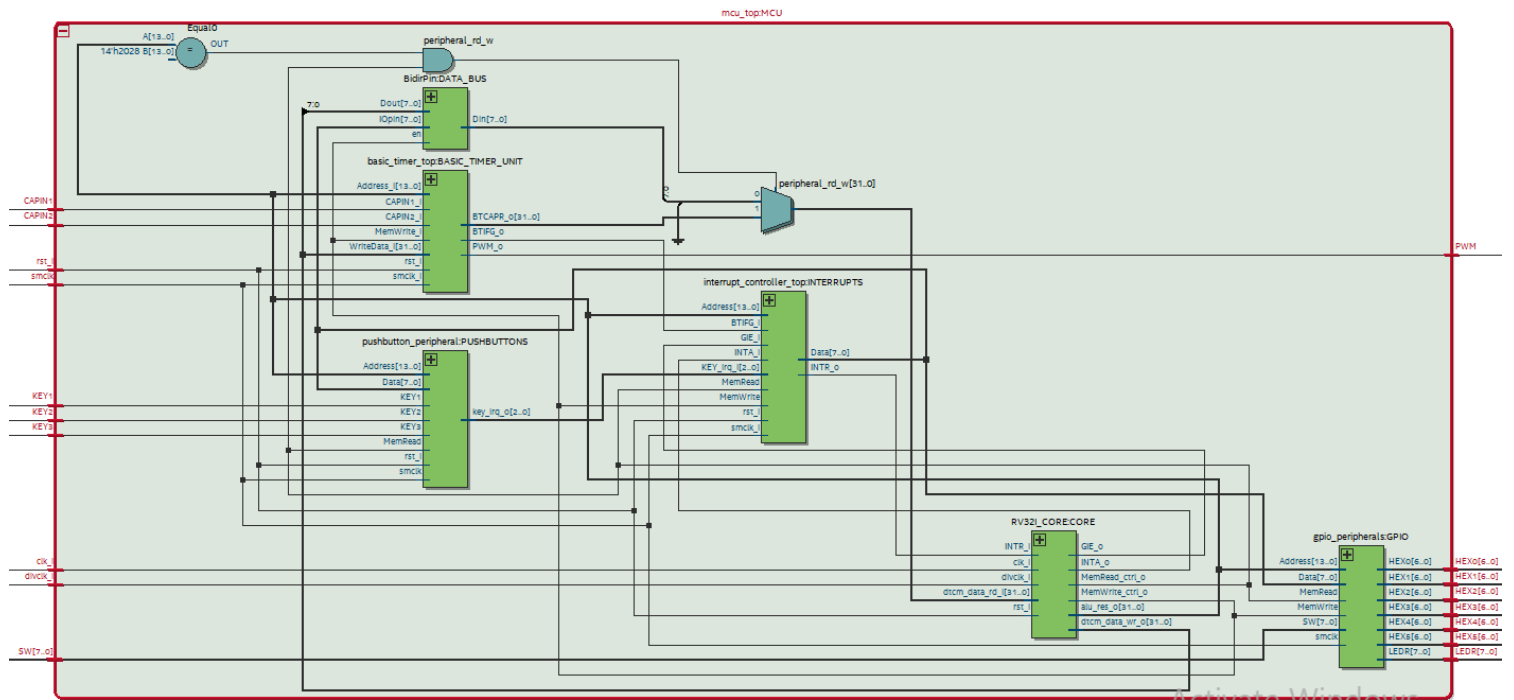
we will start by explaining each component of our design and what its purposes.

here we can see in general the RTL of the top system on our MCU that named "MCU_SYSTEM", a clock going to the "clock tree" component and 3 clk's are going out from it : {mclk, smclk, divclk} - 3 pll use for outputting the corresponding clk frequency according for the unit its going to.

The values of {mclk, smclk, divclk} in set do to the quartus f_max analysis and are reduced a bit to avoid problems that can appear due to heat and FPGA slack.

from where is entering the MCU_TOP entity.

1.1) MCU_TOP



Picture 2 : MUC_top

MCU_top is the structural container that wires the RISC-V core to every memory mapped peripheral through the BUS Interface Logic.

It holds no functional logic of its own

it the upper-level entity of all other files in our project:

bidir pin - To prevent the simultaneous transmission of two signals on the bus, while allowing the data bus to function and transfer information in both directions.

basic timer top - This is the component that unifies all the timer rates and interfaces with the interrupt unit and the processor.

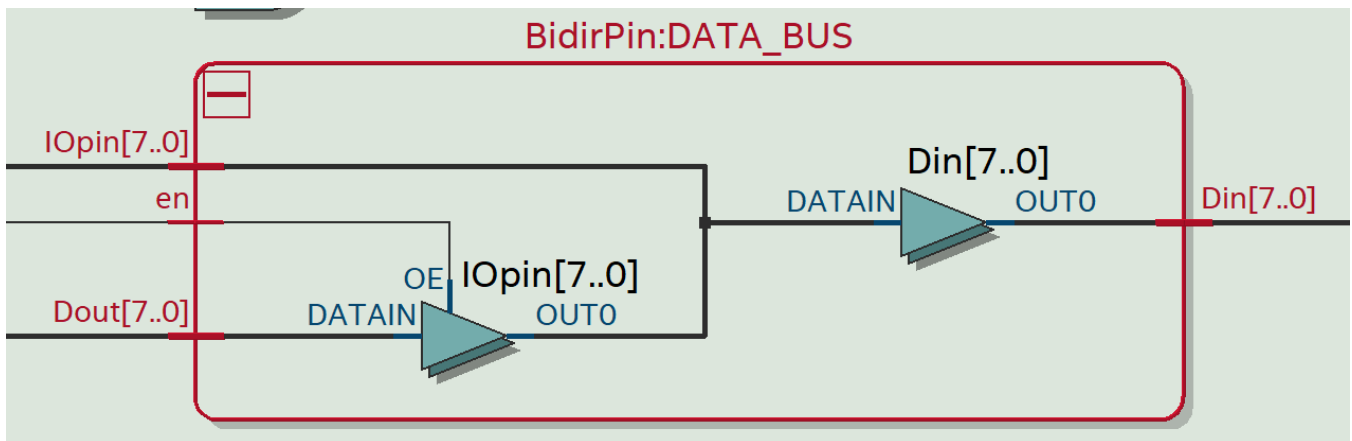
Pushbutton_periperihel - Detects button presses on KEY1–KEY3, generates interrupt requests, and allows the processor to read their states through the memory-mapped data bus

Interrupt controler top - Collects interrupt requests, applies enable and priority logic, identifies the interrupt source, and manages the interrupt handshake with the processor.

RV32-core - Executes the program instructions and performs arithmetic, memory, control-flow, multiplication, division, and interrupt-handling operations.

Gpio pariperihel - Provides the memory-mapped interface between the processor and the switches, LEDs, and seven-segment displays, supporting both input reads and output writes.

1.1.1)bidir_pin:



Picture 3 : BidirPin

BidirPin is a generic tri-state buffer, it is the component that realises the bi-directional Data BUS of the system architecture.

Dout is driven onto the shared IOpin net whenever en is high and released to high impedance otherwise, while Din continuously reads back whatever is currently present on that net.

In mcu_top en is the CPU MemWrite signal, so the core owns the bus during a store and releases it during a load, letting the addressed peripheral drive the data instead. That same release is what allows the interrupt controller to place TYPE on the bus during active-low INTA without using the address bus.

Interrupt expiation:

One of the requirements of this project was to add interrupt capability and a Basic Timer to the MCU. Interrupt capability means the CPU no longer has to keep polling the peripherals in software: a peripheral can raise a request by itself, and the hardware makes the processor stop what it is doing, jump to a service routine, and come back afterwards to exactly the instruction it left. The Basic Timer is one of those sources, and it also gives the system a sense of time, since it can count, generate a PWM output, and capture the counter value on an external event.

In our design this is implemented by three pieces working together.

The peripherals produce the requests: pushbutton_peripheral turns a press of KEY1 to KEY3 into a single-cycle event, and basic_timer raises BTIFG when its counter reaches a compare value or when a capture happens.

The interrupt controller stores these events in IFG, masks them with IE, chooses the most important one with a priority encoder, gives it a vector number in TYPE, and raises INTR to the CPU. The CPU side, inside CONTROL and IDECODE, answers with INTA, disables further interrupts, reads the handler address from the vector table and saves the return address, so the routine can run and reti can bring the program back.

The result is that software only has to write IE once and provide the handlers, and everything after that happens in hardware.

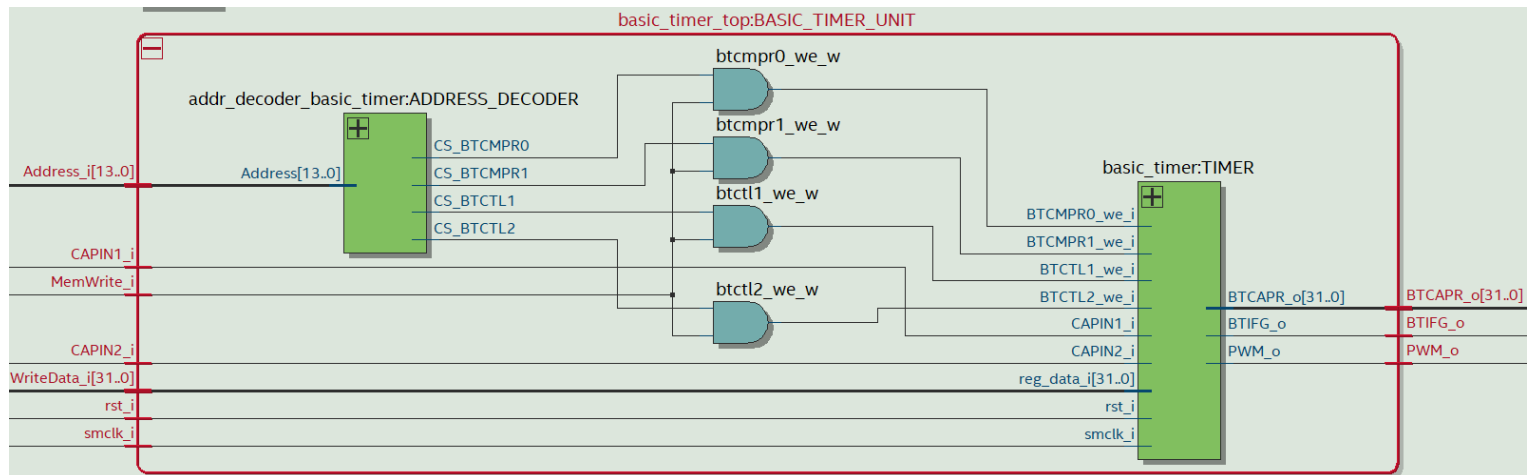
The architecture we built utilizes and addresses the aforementioned memory location.

A LOAD operation targeting the relevant address is the one that sends or receives data.

Memory-mapped Peripherals <u>with</u> interrupt capability			
I/O device name	I/O mapping address	Address Resolution	Device
PORT_PB	0x2014	Byte address	KEY [3-1]
BTCTL1	0x201C	Byte address	Basic Timer
BTCTL2	0x201D	Byte address	Basic Timer
BTCMPR0	0x2020	Word address	Basic Timer
BTCMPR1	0x2024	Word address	Basic Timer
BTCAPR	0x2028	Word address	Basic Timer
IE	0x202C	Byte address	Interrupt Controller
IFG	0x202D	Byte address	Interrupt Controller
TYPE	0x202E	Byte address	Interrupt Controller

Picture 4 : memory – mapped

1.1.2)basic_timer_top:



Picture 5 : basic_timer_top

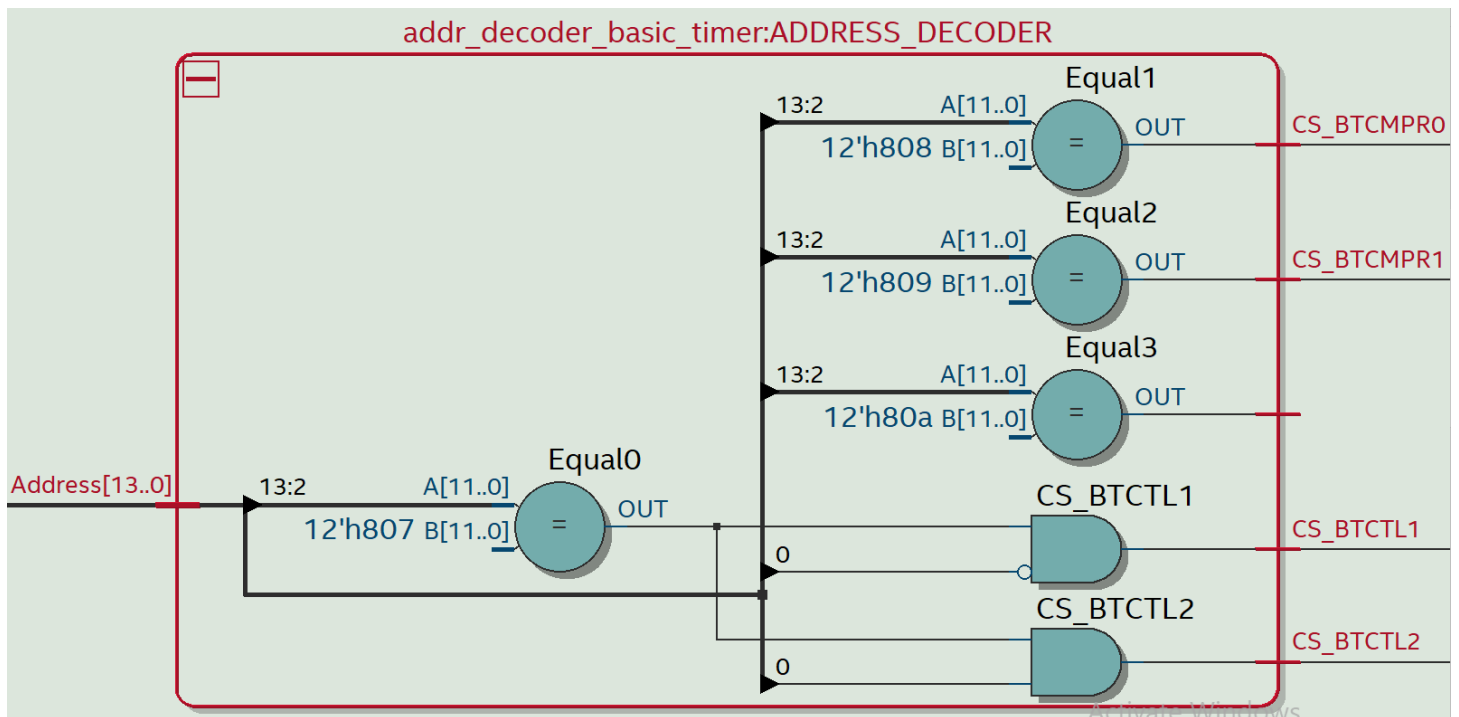
basic_timer_top is the structural wrapper of the Basic Timer, and it contains only the address decoder, the timer itself, and the write-enable gating between them. The decoder turns the shared 14-bit address into the four chip selects CS_BTCTLR1, CS_BTCTLR2, CS_BTCTLR0 and CS_BTCTLR1, and each of them is ANDed with MemWrite_i so that a timer register is loaded only during a valid CPU write to its own address.

CS_BTCTLR is deliberately left unconnected because BTCAPR is a capture-only register that hardware writes and software can only read, and its 32-bit value is returned to the core through the dedicated multiplexer in mcu_top.

The full 32-bit WriteData_i is passed straight through as reg_data_i, since BTCMPR0 and BTCMPR1 are word registers while BTCTLR1 and BTCTLR2 use only the low byte, and CAPIN1/CAPIN2 pass through untouched to the capture logic, with BTCAPR_o, BTIFG_o and PWM_o leaving as the unit's system outputs.

The two blocks inside it, addr_decoder_basic_timer and basic_timer, are described separately below.

1.1.2.1) adder_decoder_basic_timer :



Picture 6 : RTL of adder_decoder_basic_timer

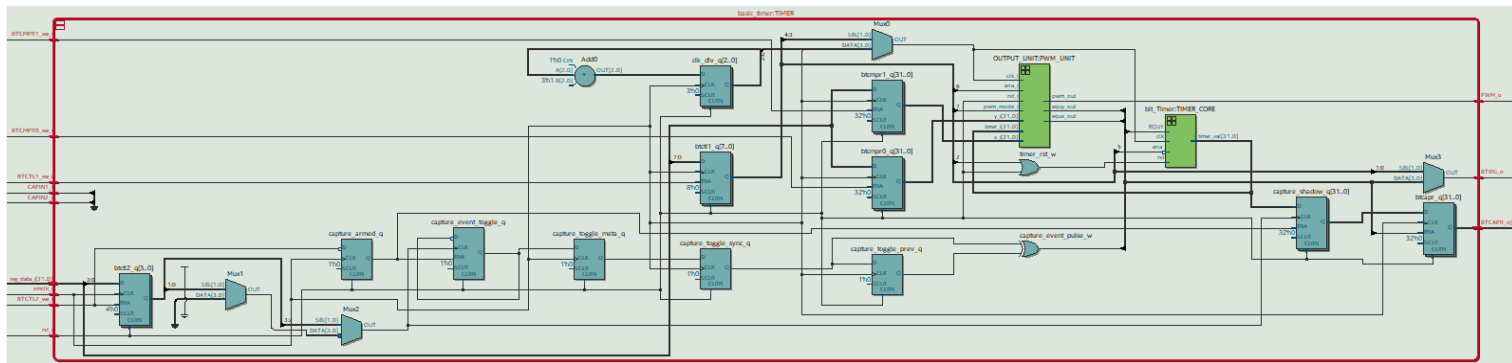
`adder_decoder_basic_timer` decodes the address bus and produces one chip select per timer register.

His general purpose is to give you information - how do we need to address it now?

It compares the upper address bits, `Address(13 downto 2)`, against `12'h807`, `12'h808`, `12'h809` and `12'h80A`, which are the byte addresses `0x201C`, `0x2020`, `0x2024` and `0x2028`.

`BTCTL1` and `BTCTL2` sit in the same word, so the `12'h807` match is split by `Address(0)`: low gives `CS_BTCTL1` and high gives `CS_BTCTL2`. `CS_BTCTRL3` is left unused, because `BTCTRL3` is written by the capture logic and read back through the separate 32-bit path in `mcu_top`.

1.1.2.2) basic_timer:



Picture 7 : RTL of basic_timer

basic_timer is the actual timer: it holds the memory-mapped registers BTCTL1, BTCTL2, BTCMPR0 and BTCMPR1, and drives the counter, the PWM output and the capture logic from them.

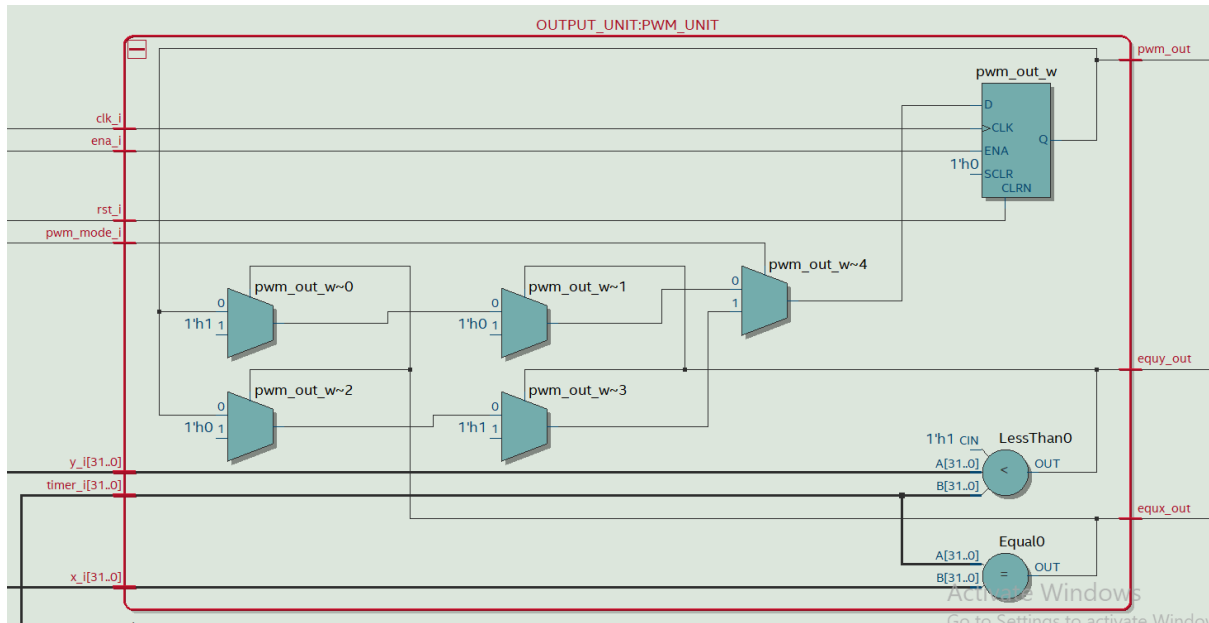
BTCTL1 selects the counter clock through a small divider (BTSSEL), holds or clears the counter (BTHOLD, BTCLR) and chooses which event raises BTIFG (BTINT), while the counter itself is the bit_Timer:TIMER_CORE block and the compare and PWM generation happen in OUTPUT_UNIT:PWM_UNIT.

BTCTL2 controls capture: two multiplexers pick the capture source (CAPIN1, CAPIN2, VCC or GND) and the active edge, and on that edge the current counter value is stored in capture_shadow_q.

Because the capture edge is asynchronous to smclk, the event is passed as a toggle bit through two synchronizer flip-flops and turned back into a one-cycle pulse, which then loads BTCAPR and can also be selected as the BTIFG source.

The blocks inside it, bit_Timer and OUTPUT_UNIT, are described separately below:

1.1.2.2.1) output_unit



Picture 8 : RTL of output_unit

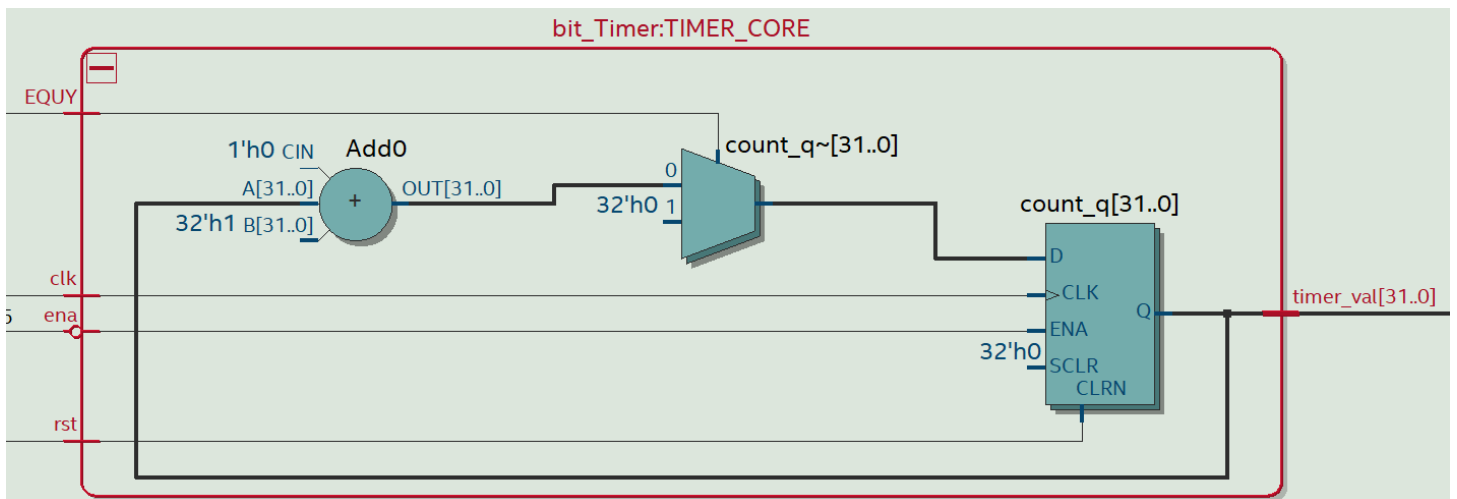
The core responsibility of the `OUTPUT_UNIT` is to compare the timer counter with the two compare registers and build the PWM signal from the result.

`equy_out` goes high when the counter reaches `BTCMPR0`, which sets the PWM period, and `equx_out` goes high when the counter equals `BTCMPR1`, which sets the duty cycle.

The flip-flop at the output turns these two events into a square wave: one match sets it and the other clears it, `pwm_mode_i` swaps which is which, and `ena_i` (`BTOUTEN`) holds the output frozen when the PWM is off.

`equy_out` is also sent back to the counter to reset it at the end of every period, and both compare signals can be chosen as the timer interrupt source.

1.1.2.2.2) bit_timer



Picture 9 : RTL of bit_timer

bit_Timer is the counter itself, a plain 32-bit up-counter.

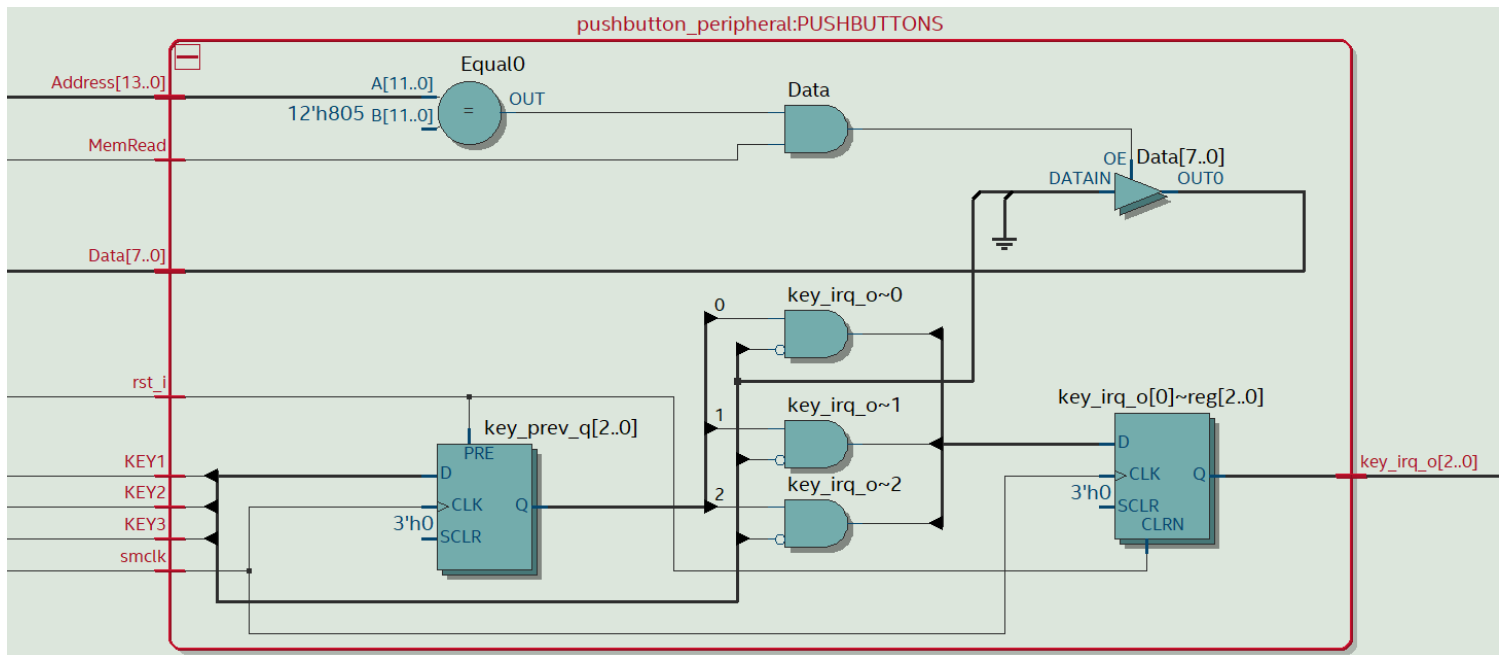
On every clock edge, if ena is high, the adder adds 1 to the current value and stores it back in count_q.

The multiplexer next to the adder handles the wrap: when EQUY comes back from the compare unit the counter is loaded with zero instead, so it restarts at the end of each period.

rst clears it asynchronously, ena comes from BTHOLD so software can freeze it, and the value leaves as timer_val, which is BTCNT.

This unit is working with the OutPut_Unit.

1.1.3) pushbutton_periperihel



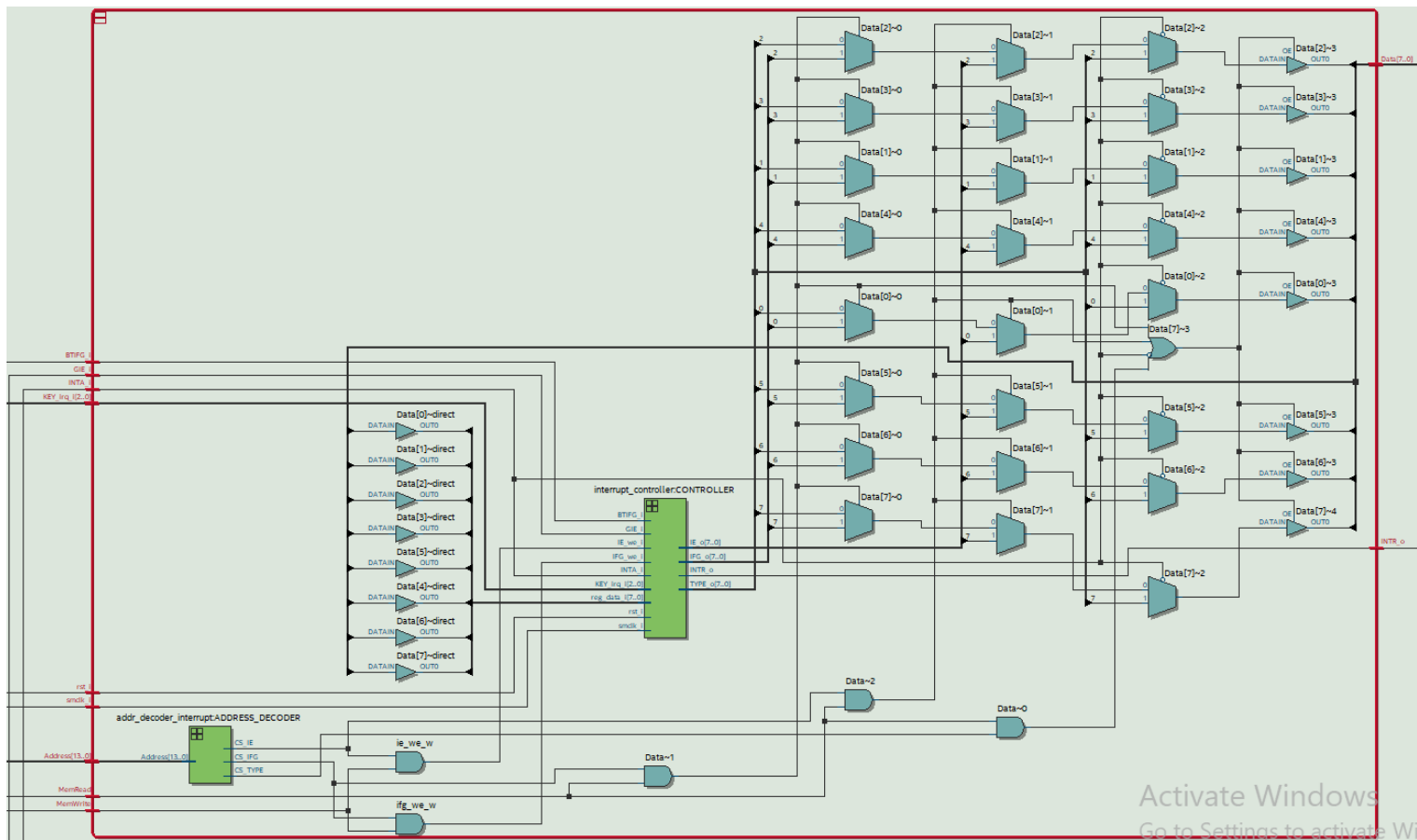
Picture 10 : RTL of PushButton_Periperihel

pushbutton_peripheral handles the three pushbuttons KEY1 to KEY3 and does two separate jobs.

- 1) is the read port: Equal0 compares the address against 12'h805, which is byte address 0x2014 (PORT_PB), and together with MemRead it enables a tri-state driver that puts the current button levels on the data bus, so software can read the buttons directly.
- 2) is edge detection for interrupts: key_prev_q samples the three buttons on every smclk edge, and the AND gates compare the previous sample with the current level to detect a high-to-low transition, since the buttons are active low.

The result is registered again into key_irq_o, so each press produces a single one-cycle pulse per button, and these three pulses are what set the KEY1, KEY2 and KEY3 flags in the interrupt controller.

1.1.4) interrupt_controller_top



Picture 11 : RTL of interrupt controller top

interrupt_controller_top is the box that connects the interrupt controller to the shared bus. The address decoder checks which of the three registers the CPU is talking to, IE, IFG or TYPE, and the two AND gates make sure a register is only written when the CPU is really doing a write.

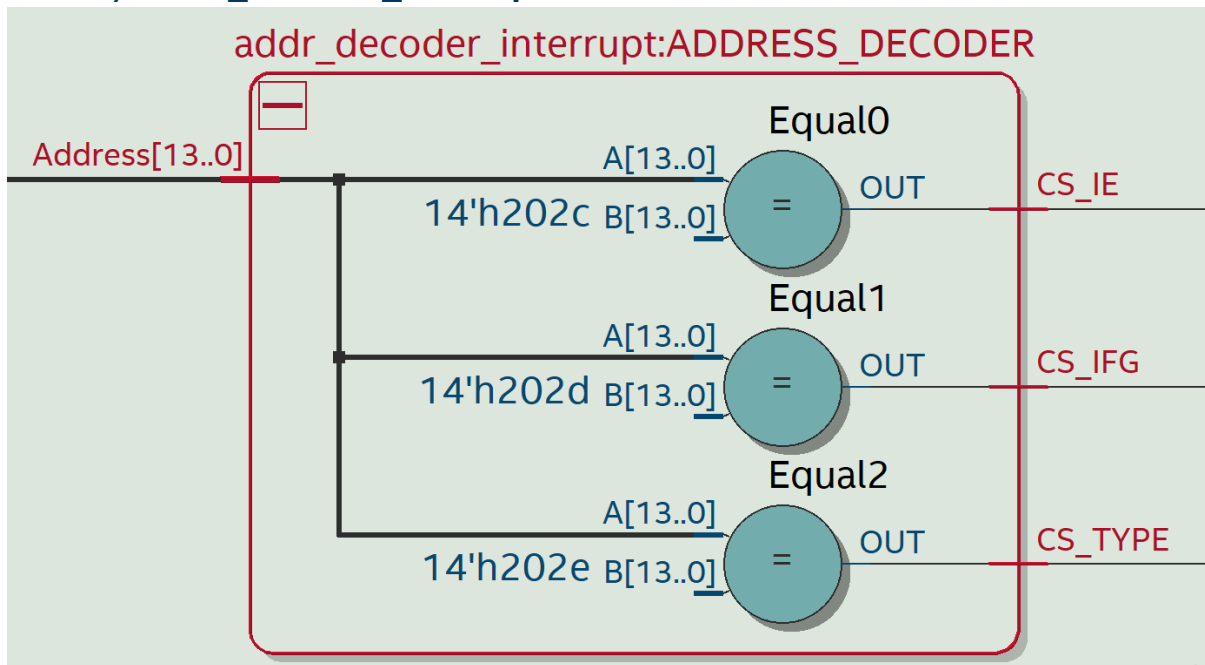
The buffers on the left let the CPU's data come into the block, so software can set IE and IFG.

The multiplexers on the right choose which of the three registers is sent back to the CPU, and TYPE always wins when INTA is low, so the CPU gets the interrupt number during the acknowledge.

The tri-state buffers at the edge let the block release the bus when it is not being used, so other peripherals can drive it instead.

The block inside it “addr_decoder_interrupt” and “interrupt_controller” is described separately below.

1.1.4.1) addr_decoder_interrupt



Picture 12 : RTL of interrupt_controller_top

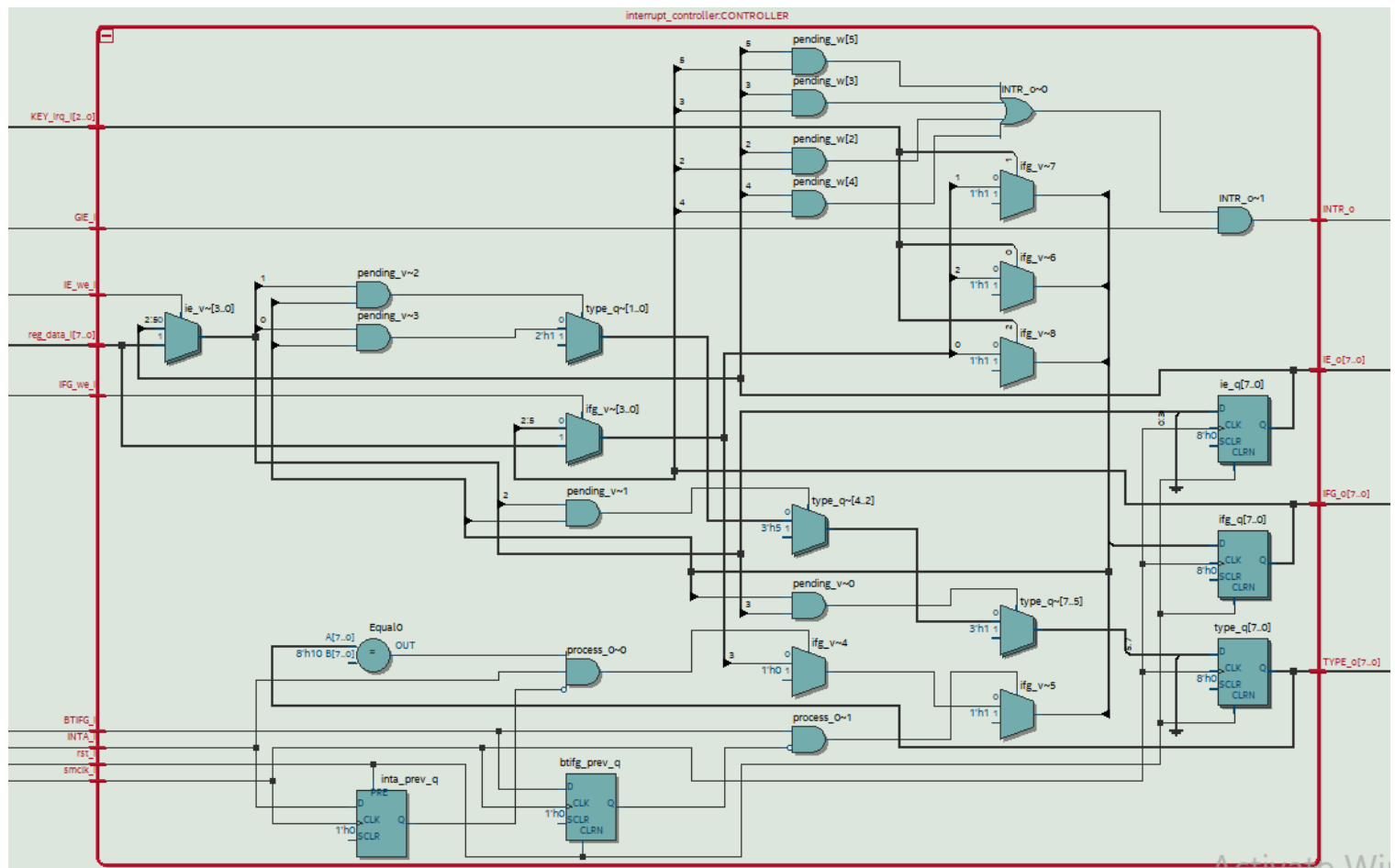
addr_decoder_interrupt tells the interrupt controller which of its registers the CPU is addressing.

It takes the 14-bit address and compares it with three fixed values, 0x202C for IE, 0x202D for IFG and 0x202E for TYPE, using one comparator each.

Only one of the three outputs can be high at a time, since the addresses are different.

These chip selects are then used outside the block, together with MemWrite and MemRead, to decide which register is written and which one is placed back on the data bus.

1.1.4.2) interrupt_controller



Picture 13 : RTL of interrupt_controller

interrupt_controller is the block that decides if and which interrupt the CPU should take.

It holds three registers:

- 1) `ie_q`, which software writes to enable each source.
- 2) `ifg_q`, which stores a flag whenever a source fires.
- 3) `type_q`, which holds the vector number of the interrupt being served.

IE, Interrupt Enable Register

7	6	5	4	3	2	1	0
0	0	KEY3IE	KEY2IE	KEY1IE	BTIE	TXIE	RXIE
r-0	rw	rw	rw	rw	rw	rw	rw

IEx Bit x 0 Interrupt not enabled
1 Interrupt enabled

IFG, Interrupt Flag Register

7	6	5	4	3	2	1	0
0	0	KEY3IFG	KEY2IFG	KEY1IFG	BTIFG	TXIFG	RXIFG
r-0	rw	rw	rw	rw	rw	rw	rw

IFGx Bit x 0 No interrupt pending
1 Interrupt pending

TYPE, Interrupt Type Register

7	6	5	4	3	2	1	0
0	0	TYPEx					
r-0	r-0	r	r	r	r	r-0	r-0

Picture 14 : IE, IFG, TYPE Register

The multiplexers around ifg_q are the ways a flag can change: a source event sets it, a software writer can set or clear it, and the timer flag is also cleared automatically after the acknowledge, with the source event given priority so a new request is never lost.

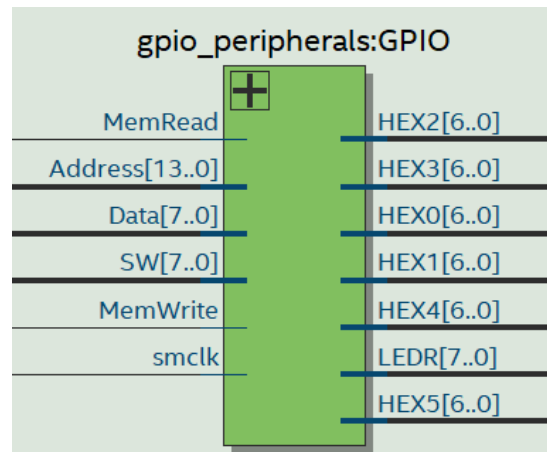
The AND gates named pending are simply ie_q and ifg_q bit by bit, meaning a source that is both enabled and flagged, and the OR gate combines them with GIE to produce INTR_o, the request line to the CPU. The chain of multiplexers feeding type_q is a priority encoder: it picks 0x10 for the Basic Timer, then 0x14, 0x18 and 0x1C for KEY1, KEY2 and KEY3, so if several interrupts are pending at once the CPU is given the most important one first.

The two small flip-flops at the bottom, inta_prev_q and btifg_prev_q, remember the previous value of INTA and BTIFG so the block can react to edges rather than levels.

Basic Timer	BTIFG	10h	maskable	4
Pushbutton 1	KEY1	14h	maskable	5
Pushbutton 2	KEY2	18h	maskable	6
Pushbutton 3	KEY3	1Ch	maskable	7, Lowest

Picture 15 : the DTCM eldresses list that store where we need to jump to -corresponding to etch Service ration

1.1.5) gpio_pheriperihal :

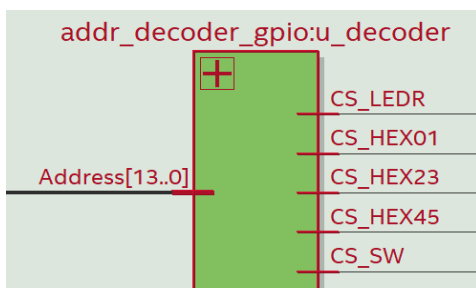


Picture 16 : RTL of Gpio_pheriperihal

gpio_peripherals is the block that controls the simple things on the board: the red LEDs, the six digit displays and the slide switches. It is built from four small parts.

in dis file we have 4 sub files:

1- addr_decoder_gpio: looks at the address and decides which port the CPU is talking to, the LEDs, one of the displays, or the switches. Its outputs are combined with MemWrite to make a write signal, and with MemRead to make a read signal. corresponding to the fowling table



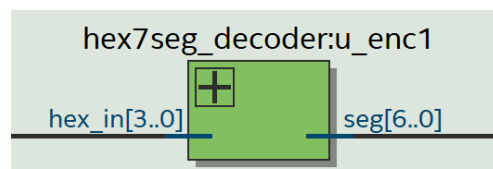
Picture 17 : RTL of adder_secoder_pgio

one instance. It compares the address bits against five fixed values and decides which port the CPU is talking to: the LEDs at 0x2000, the HEX0/HEX1 pair, the HEX2/HEX3 pair, the HEX4/HEX5 pair, or the switches at 0x2010. Each pair shares one chip select because the two ports sit in the same word, so address bit 0 separates them afterwards.

Memory-mapped GPIO <u>without</u> interrupt capability				
I/O device name	I/O mapping address	Note	Address Resolution	Direction
PORT_LEDR	0x2000	LEDR7- LEDR0	Byte address	GPO
PORT_HEX0	0x2004		Byte address	GPO
PORT_HEX1	0x2005		Byte address	GPO
PORT_HEX2	0x2008		Byte address	GPO
PORT_HEX3	0x2009		Byte address	GPO
PORT_HEX4	0x200C		Byte address	GPO
PORT_HEX5	0x200D		Byte address	GPO
PORT_SW	0x2010	SW7- SW0	Byte address	GPI

Picture 18 : Gpio map table

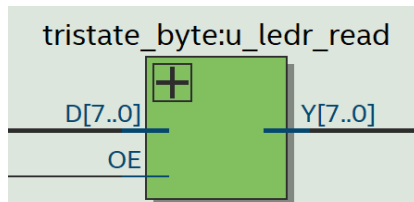
2 - hex7seg_decoder,



Picture 19 : hex7seg_decoder

six instances. Each one takes the low four bits from its latch and converts that hexadecimal digit into the seven segment pattern that lights up one display.

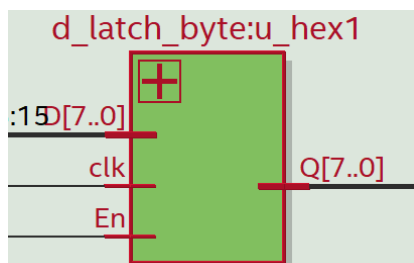
3- tristate_byte,



Picture 20 : tristate_byte

eight instances. These are what allow reading over the shared bidirectional data bus. Seven of them send back the byte currently stored in the LED and display latches, so software can read a port it wrote earlier, and the eighth places the switch inputs SW7 to SW0 on the bus. Each driver turns on only when its own read enable is high and is released otherwise, so only one source drives the bus at a time.

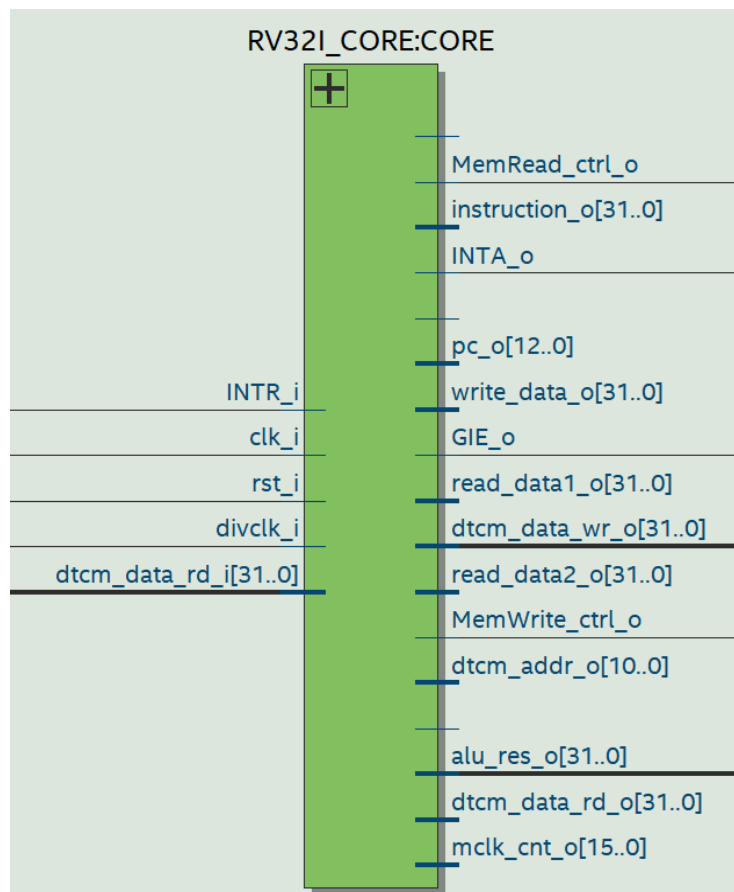
4- d latch byte,



Picture 21 : d_latch_byte

seven instances. Each one is an 8-bit storage element that keeps the byte the CPU wrote, one for the LEDs and one for every display digit, so the value stays on the board until the next write. The latch opens only while smclk is low, because that is when the address bus is stable and the byte is guaranteed to land in the correct port.

1.1.6) RV32-CORE :



Picture 22 :RTL of RV32-CORE

RV32I_CORE is the CPU. It is a single-cycle RV32IM processor, built from smaller blocks: fetch, decode, execute, control, data memory, a multiplier and a division accelerator.

Its inputs are the CPU clock, the faster clock for the divider, the reset, the data coming back from a peripheral, and INTR_i, the interrupt request.

Its main outputs are the address it wants to access, the data it wants to store, and two signals that say whether this is a read or a write. These four are what the bus in mcu_top uses to reach every peripheral.

INTA_o and GIE_o are the interrupt handshake: INTA answers the controller, and GIE says whether interrupts are allowed right now.

All the other outputs, like pc_o and instruction_o, are not used by the design. They are brought out only so we can watch them in Signal Tap and measure performance.

To support interrupts, changes were added to some of the blocks, while others are reused as they are.

IFETCH: no new logic was added. The interrupt reuses the PC hold input, which freezes the program counter for one cycle, and the JALR target input, which is used as a fake jump to the handler address.

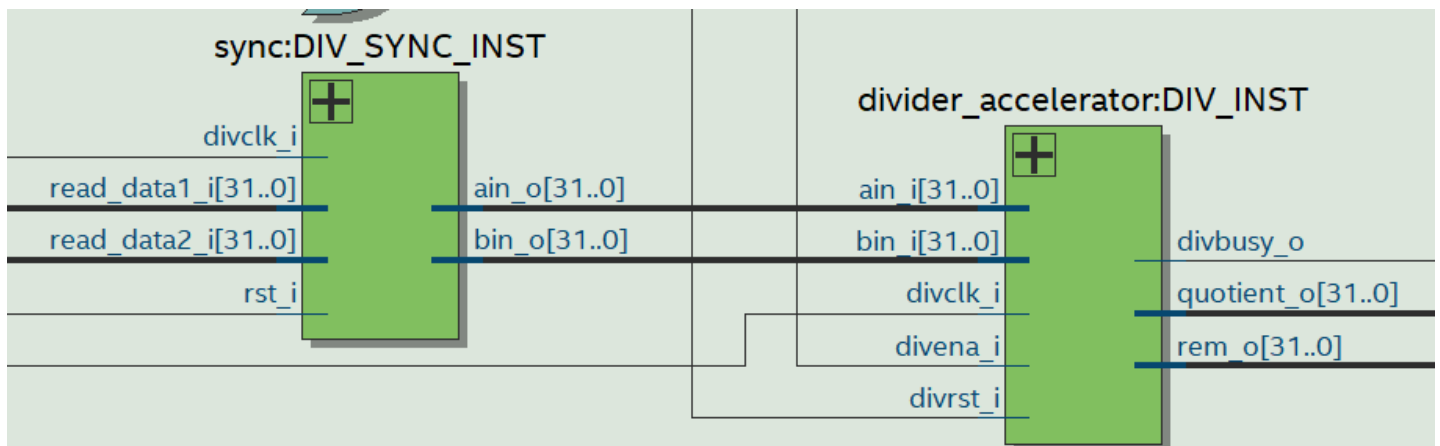
IDECODE: two registers were given a special role. GIE is bit 0 of gp (x3), so the hardware can clear it when an interrupt starts and set it again on reti, and it is also sent out to the interrupt controller. The return address is written into tp (x4), which is what reti jumps back to.

CONTROL: a small state machine was added. It waits for INTR, then runs two cycles: in the first it sends INTA, freezes the PC and clears GIE, and in the second it latches TYPE and starts the jump. It also blocks all control signals during those two cycles, so the instruction that was fetched does nothing.

EXECUTE: no change. The ALU is not involved in the interrupt sequence, because the handler address comes from memory and not from a calculation.

DMEMORY: no change. It is simply given TYPE as its address during the second cycle, so it reads the vector table entry that holds the address of the handler.

1.1.7) sync and divider_accelerator



Picture 23 :RTL of sync and divider_accelerator

sync is the bridge between the two clocks.

NOTE that an entry of `read_data1` and `read_data2` are from enter 2 registers that are in the rv32-core and controlled by `mclk`.

The CPU and the divider operate with different clocks.

The CPU uses the slower MCLK, while the divider uses the faster DIVCLK.

Therefore, the two operands pass through two flip-flop stages before entering the divider. This gives the signals enough time to become stable in the fast-clock domain. The synchronized outputs `ain_o` and `bin_o` are used as the dividend and divisor.

The divider_accelerator performs unsigned division using the restoring-division algorithm. `divrst_i` loads the operands and initializes the divider, while `divena_i` starts the operation.

The divider calculates one quotient bit on every rising edge of DIVCLK, so a complete 32-bit division requires 32 fast-clock cycles. While it is working, `divbusy_o` remains high and tells the CPU to wait. When the operation finishes, `quotient_o` contains the division result and `rem_o` contains the remainder.

The divider also interacts with the interrupt mechanism. If an interrupt arrives while a division is running, the processor waits until the division finishes before accepting it. This prevents the division instruction from being interrupted, restarted, or completed with incorrect operands. Both mechanisms can hold the program counter through the PC-hold logic.

As a result, interrupt response may be delayed by up to one complete division operation.

how interrupt flow in the design:

1 - The press. KEY1 to KEY3 are active low. In pushbutton_peripheral the three buttons are sampled on every smclk edge into key_prev_q, and comparing the previous sample with the current level produces a single one-cycle pulse on the falling edge, on key_irq_o.

2- The flag is stored. In interrupt_controller that pulse sets the matching bit of ifg_q, bit 3 for KEY1, bit 4 for KEY2 and bit 5 for KEY3. The flag stays set until someone clears it, so a short press is never lost.

3- The request is formed. The controller computes pending as ie_q AND ifg_q, so only sources the software enabled through IE can pass. A priority encoder loads type_q with the vector number of the most important pending source, 0x10 for the Basic Timer and 0x14, 0x18, 0x1C for KEY1, KEY2 and KEY3, and INTR_o goes high if anything is pending and GIE is set.

4- The CPU accepts. In CONTROL the interrupt state machine leaves idle only if INTR is high and the divider is not stalling, so a division in progress always finishes first.

5- First cycle, acknowledge. CONTROL drives INTA low, freezes the program counter through the PC hold input of IFETCH, and asks IDECODE to clear GIE, which is bit 0 of gp (x3), so no second interrupt can enter. Because INTA is low, interrupt_controller_top gives TYPE the highest priority on the data bus and drives it without any address, and CONTROL latches it. All control signals are blocked in this cycle, so the instruction that was fetched does nothing.

6- Second cycle, service. RV32I_CORE replaces the data address with the zero extended TYPE value, so DMEMORY reads the vector table entry at Memory[TYPE], which holds the address of the handler. In the same cycle IDECODE writes the return PC into tp (x4), and the core reuses the JALR path of IFETCH with that memory read as the target, so the next instruction fetched is the first instruction of the handler.

7- The handler runs. From here everything is ordinary instruction execution. GIE is zero, so the routine cannot be interrupted, and before returning it clears its own flag by reading IFG, masking the bit and writing the value back. That write passes the address decoder and the MemWrite gate in interrupt_controller_top and reaches ifg_q, which makes pending drop and INTR go low.

8- The return. reti is simply jalr zero, 0(tp), so it is executed as a normal jump through EXECUTE and IFETCH and the PC returns to the instruction that was interrupted. CONTROL detects it while in the idle state and asks IDECODE to set GIE back to one, so the next interrupt can be accepted.

2) PPA Characterization of RV32IM-Based MCU Design

Area						
	Logic utilization (ALMs)	Registers	I/O pins	Embedded memory bits	DSP blocks	PLLs
MCU with GPIO	1,779 / 41,910 (4%)	1,796	66 / 499 (13%)	131,072 / 5,662,720 (2%)	3 / 112 (3%)	2 / 15 (13%)
MCU with GPIO & interrupt	1,916 / 41,910 (5%)	1,811	66 / 499 (13%)	131,072 / 5,662,720 (2%)	3 / 112 (3%)	2 / 15 (13%)

Performance			
	fmax	Critical path (slowest submodule and reason)	fMCLK (= fsysclk)
MCU with GPIO	31.71 MHz	Divider accelerator dividend register → accelerator write-back multiplexers → Register File write port (DIV_INST dividend_q[36] → Idecode:ID RF_q[22][4]). DIVCLK-launched / MCLK-captured cross-domain transfer. Setup slack = 0.411 ns.	25 MHz
MCU with GPIO & interrupt	29.87 MHz	Divider accelerator dividend register → accelerator write-back multiplexers → Register File write port (DIV_INST dividend_q[46] → Idecode:ID RF_q[4][14]). DIVCLK-launched / MCLK-captured cross-domain transfer with an 8.000 ns launch-to-latch relationship. Setup slack = 0.130 ns (second-worst path 0.353 ns).	25 MHz

Power				
	Total power consumption	Static power consumption	Dynamic power consumption	I/O power consumption
MCU with GPIO	470.89 mW	419.23 mW	40.95 mW	10.71 mW
MCU with GPIO & interrupt	470.14 mW	419.22 mW	40.21 mW	10.71 mW

2.1) Supporting Quartus Reports - WITH INTERRUPTS

Flow Status	Successful - Sun Aug 30 15:15:42 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	LAB_5_PIP
Top-level Entity Name	mcu_system
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	1,916 / 41,910 (5 %)
Total registers	1811
Total pins	66 / 499 (13 %)
Total virtual pins	0
Total block memory bits	131,072 / 5,662,720 (2 %)
Total DSP Blocks	3 / 112 (3 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	2 / 15 (13 %)
Total DLLs	0 / 4 (0 %)

Figure 24 – Quartus Prime Flow Summary: MCU with GPIO and interrupt capability (revision LAB_5_PIP, Cyclone V 5CSXFC6D6F31C6 – 1,916 ALMs, 1,811 registers).

	Fmax	Restricted Fmax	Clock Name	Note
1	29.87 MHz	29.87 MHz	CLKTREE\G0:MCLK_PLL_U\altpll_component\auto_generated\generic_pll1~PLL_OUTPUT_COUNTER\divclk	
2	86.41 MHz	86.41 MHz	altera_reserved_tck	
3	152.93 MHz	152.93 MHz	CLKTREE\G0:DIVCLK_PLL_U\altpll_component\auto_generated\generic_pll1~PLL_OUTPUT_COUNTER\divclk	

Figure 25 – Slow-model clock summary, interrupt-enabled build: Fmax per clock domain (MCLK PLL divclk = 29.87 MHz, DIVCLK PLL divclk = 152.93 MHz, altera_reserved_tck = 86.41 MHz).

Slow 1100mV 85C Model

Command Info		Summary of Paths				
Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	
1 0.130	mcu_top:MCU[...vidend_q[46]	mcu_top:MCU[RV3...ID]RF_q[4][14]	CLKTREE\G0:DIVCLK_PLL..._OUTPUT_COUNTER\divclk	CLKTREE\G0:MCLK...UT_COUNTER\divclk	8.000	

Path #1: Setup slack is 0.130

Path Summary		Statistics	Data Path	Waveform	Extra Fitter Information
Property		Value			
1	From Node	mcu_top:MCU[RV32I_CORE:CORE]divider_accelerator:DIV_INST[dividend_q[46]			
2	To Node	mcu_top:MCU[RV32I_CORE:CORE]decoder:ID[RF_q[4][14]			
3	Launch Clock	CLKTREE\G0:DIVCLK_PLL_U\altpll_component\auto_generated\generic_pll1~PLL_OUTPUT_COUNTER\divclk			
4	Latch Clock	CLKTREE\G0:MCLK_PLL_U\altpll_component\auto_generated\generic_pll1~PLL_OUTPUT_COUNTER\divclk			
5	Data Arrival Time	41.006			
6	Data Required Time	41.136			
7	Slack	0.130			

Figure 26 – Critical path summary: worst-case setup slack 0.130 ns, divider accelerator dividend_q[46] (DIVCLK) → register file RF_q[4][14] (MCLK), 8.000 ns clock relationship.

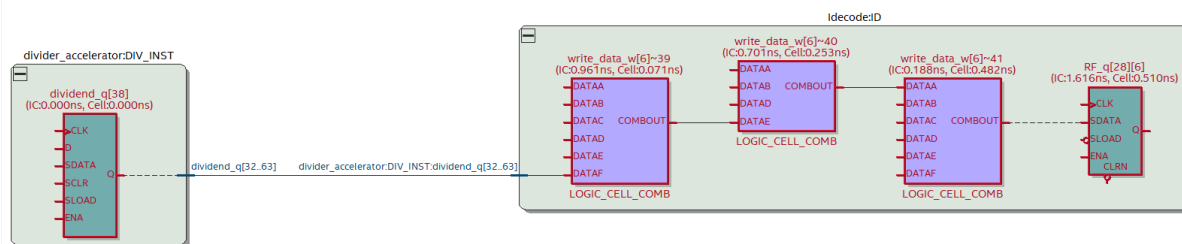


Figure 27 – RTL/technology-map view of the divider-accelerator datapath along the reported critical path.

The worst-case path starts at the divider accelerator's dividend register (divider_accelerator:DIV_INST|dividend_q[46]), passes through the divider's quotient/remainder selection and result-multiplexing logic, and terminates at the register-file write port in the decode stage (Idcode:ID|RF_q[4][14]). It is a cross-domain path: the source register is launched by the DIVCLK PLL output while the destination is captured by the MCLK PLL output, giving a launch-to-latch relationship of only 8.000 ns. With a data arrival time of 41.006 ns against a required time of 41.136 ns, the remaining setup slack is 0.130 ns, making the divider result transfer the tightest timing constraint in the interrupt-enabled build. The second-worst path is the same structure one bit-slice over (dividend_q[38] → RF_q[28][6]) with 0.353 ns slack, confirming that the whole divider write-back bus, and not a single bit, is the limiter.

Slow 1100mV 85C Model				
Command Info		Summary of Paths		
	Slack	From Node	To Node	Launch Clock
1	0.353	mcu_top:MCU RV32I_CORE:CO...r:DIV_INST dividend_q[38]	mcu_top:MCU RV32I_CORE:CORE Idcode:ID RF_q[28][6]	CLKTREE G0:DIVCLK_PLL_U...LL_OUTPUT_COL
Path #1: Setup slack is 0.353				
Path Summary		Statistics	Data Path	Waveform
Property		Value		
1	From Node	mcu_top:MCU RV32I_CORE:CO...r:DIV_INST dividend_q[38]		
2	To Node	mcu_top:MCU RV32I_CORE:CORE Idcode:ID RF_q[28][6]		
3	Launch Clock	CLKTREE G0:DIVCLK_PLL_U...LL_OUTPUT_COUNTER divclk		
4	Latch Clock	CLKTREE G0:MCLK_PLL_U a...LL_OUTPUT_COUNTER divclk		
5	Data Arrival Time	40.787		
6	Data Required Time	41.140		
7	Slack	0.353		

Figure 28 – Second-worst setup path: slack 0.353 ns, divider accelerator dividend_q[38] (DIVCLK) → register file RF_q[28][6] (MCLK); arrival 40.787 ns vs. required 41.140 ns.

Power Analyzer Summary	
 <<Filter>>	
Power Analyzer Status	Successful - Sun Aug 30 15:20:36 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	LAB_5_PIP
Top-level Entity Name	mcu_system
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	470.14 mW
Core Dynamic Thermal Power Dissipation	40.21 mW
Core Static Thermal Power Dissipation	419.22 mW
I/O Thermal Power Dissipation	10.71 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Figure 29 – Power Analyzer Summary, interrupt-enabled build: 470.14 mW total (419.22 mW static, 40.21 mW dynamic, 10.71 mW I/O); confidence Low.

2.2) MCU WITH GPIO ONLY (NO INTERRUPTS)

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun Aug 30 15:44:29 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	LAB_5_PIP
Top-level Entity Name	mcu_system
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	1,779 / 41,910 (4 %)
Total registers	1796
Total pins	66 / 499 (13 %)
Total virtual pins	0
Total block memory bits	131,072 / 5,662,720 (2 %)
Total DSP Blocks	3 / 112 (3 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	2 / 15 (13 %)
Total DLLs	0 / 4 (0 %)

Figure 30 - Quartus Prime Flow Summary: MCU with GPIO, no interrupt capability (1,779 ALMs, 1,796 registers).

LAB_5_PIP/LAB_5_PIP - LAB_5_PIP				
ts	Script	Tools	Window	Help
Slow 1100mV 85C Model				
	Fmax	Restricted Fmax	Clock Name	Note
1	31.71 MHz	31.71 MHz	CLKTREE\G0:MCLK_PLL_U\altpll_component\auto_generated\generic_pll1~PLL_OUTPUT_COUNTER\divclk	
2	97.3 MHz	97.3 MHz	altera_reserved_tck	
3	156.52 MHz	156.52 MHz	CLKTREE\G0:DIVCLK_PLL_U\altpll_component\auto_generated\generic_pll1~PLL_OUTPUT_COUNTER\divclk	

Figure 31 - Slow-model clock summary: Fmax per clock domain (MCLK PLL divclk = 31.71 MHz, DIVCLK PLL divclk = 156.52 MHz, altera_reserved_tck = 97.3 MHz).

Path #1: Setup slack is 0.411		
Path Summary	Statistics	Data Path
Property	Value	
1 From Node	mcu_top:MCU RV32I_CORE:CORE divider_accelerator:DIV_INST dividend_q[36]	
2 To Node	mcu_top:MCU RV32I_CORE:CORE decode:ID RF_q[22][4]	
3 Launch Clock	CLKTREE\G0:DIVCLK_PLL_U\altpll_component...d\generic_pll1~PLL_OUTPUT_COUNTER\divclk	
4 Latch Clock	CLKTREE\G0:MCLK_PLL_U\altpll_component...d\generic_pll1~PLL_OUTPUT_COUNTER\divclk	
5 Data Arrival Time	40.726	
6 Data Required Time	41.137	
7 Slack	0.411	

Figure 32 - Critical path summary: worst-case setup slack 0.411 ns, divider accelerator dividend_q[36] (DIVCLK) → register file RF_q[22][4] (MCLK); arrival 40.726 ns vs. required 41.137 ns.

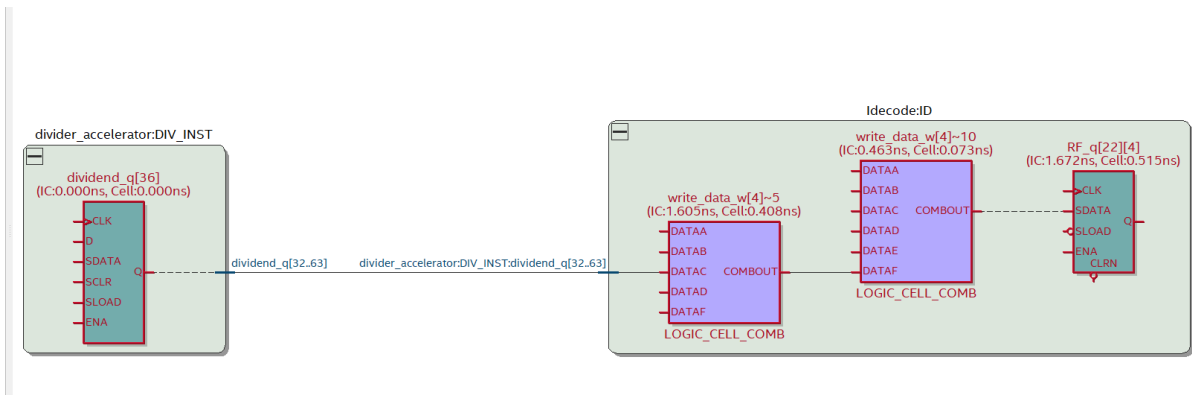


Figure 33 - RTL/technology-map view of the divider-accelerator write-back datapath along the reported critical path.

Power Analyzer Summary	
<<Filter>>	
Power Analyzer Status	Successful - Sun Aug 30 15:47:35 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	LAB_5_PIP
Top-level Entity Name	mcu_system
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	470.89 mW
Core Dynamic Thermal Power Dissipation	40.95 mW
Core Static Thermal Power Dissipation	419.23 mW
I/O Thermal Power Dissipation	10.71 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Figure 34 - Power Analyzer Summary, no-interrupt build: 470.89 mW total (419.23 mW static, 40.95 mW dynamic, 10.71 mW I/O); confidence Low.

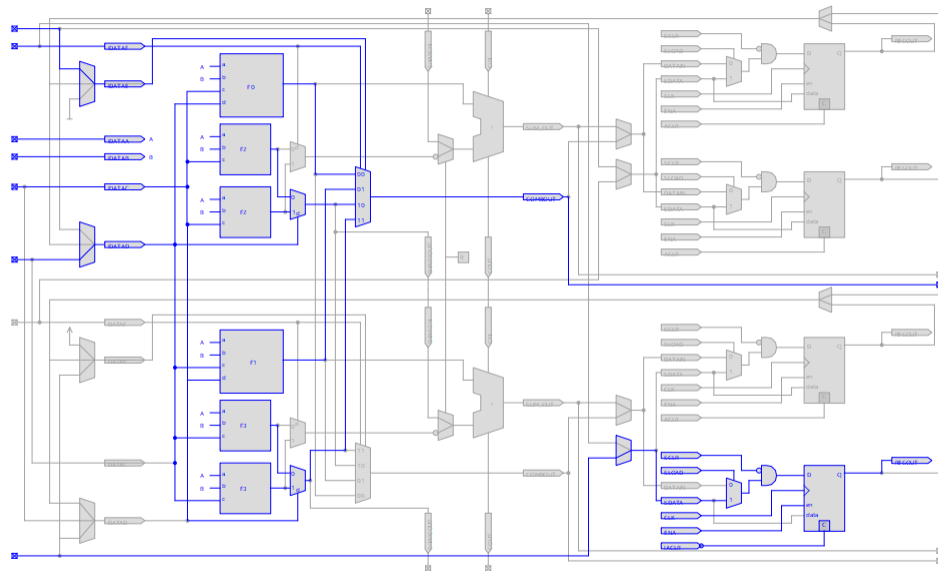
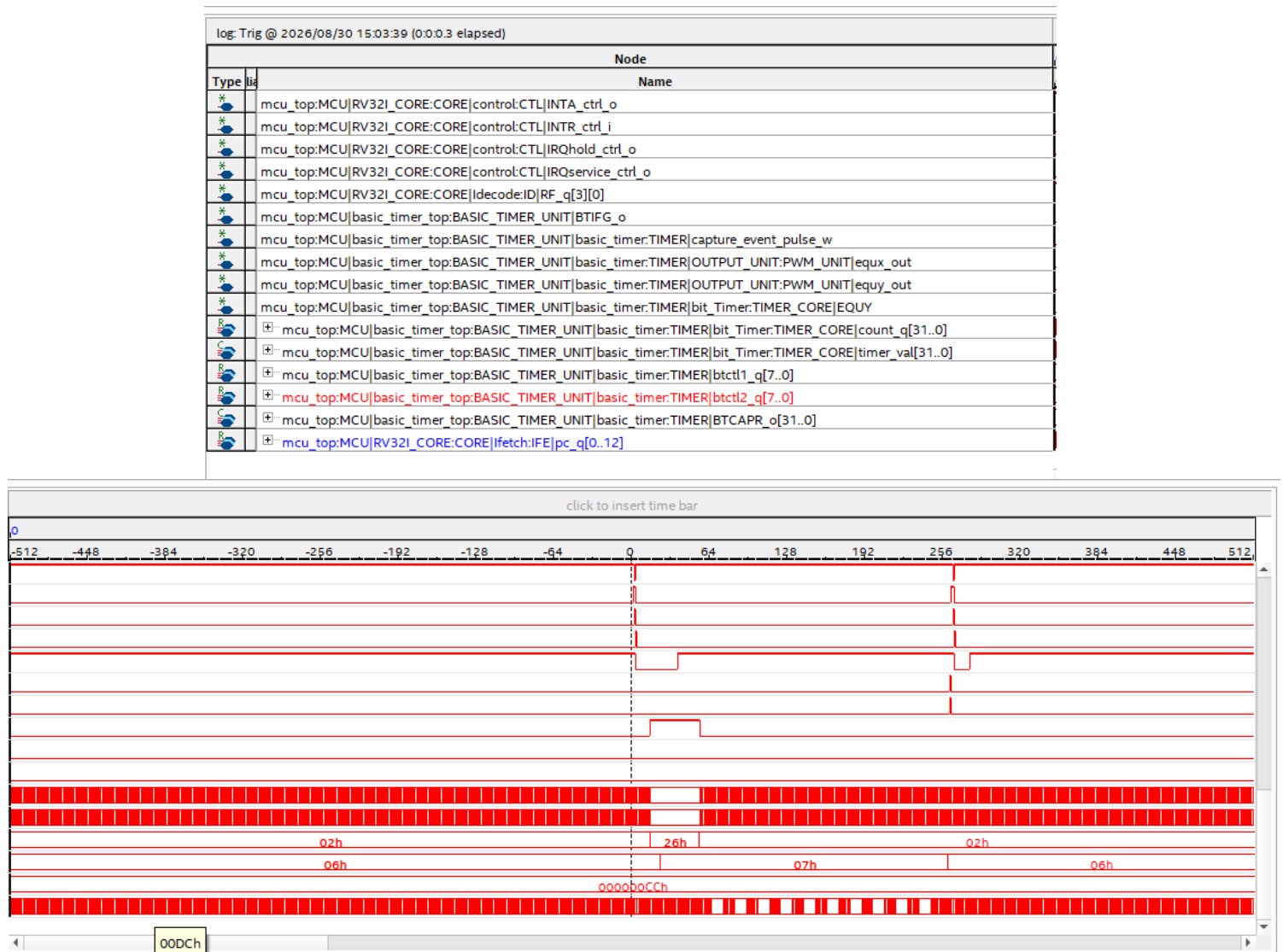


Figure 35 - RTL/technology-map view of the divider-accelerator datapath along the reported critical path.

The worst-case path starts at the divider accelerator's dividend register (divider_accelerator:DIV_INST|dividend_q[46]), passes through the divider's quotient/remainder selection and result-multiplexing logic, and terminates at the register-file write port in the decode stage (ldecode:ID|RF_q[4][14]). It is a cross-domain path: the source register is launched by the DIVCLK PLL output while the destination is captured by the MCLK PLL output, giving a launch-to-latch relationship of only 8.000 ns. With a data arrival time of 41.006 ns against a required time of 41.136 ns, the remaining setup slack is 0.130 ns, making the divider result transfer the tightest timing constraint in the interrupt-enabled build.

3) signal tap use:



SignalTap was used to monitor the MCU's internal signals directly on the FPGA. It captured the interrupt protocol, timer activity, program counter, and control signals in real time. A trigger was configured to start the capture when a selected interrupt event occurred. The recorded waveforms helped verify the correct timing and operation of the complete hardware system.

This screenshot is from test4 of inturprt :

After KEY3 is pressed, the interrupt controller starts the interrupt sequence: INTR is detected, INTA acknowledges it, IRQhold freezes the PC, and IRQservice remains active while the ISR runs. The ISR selects State 3 and configures the Basic Timer for capture mode.

The timer is then cleared and counts during the division/remainder routine. When the routine ends, a capture event copies the current counter value into BTCAPR.

The displayed value 0xCC equals 204 SMCLK cycles, representing the measured execution time; at 25 MHz, this is approximately **8.16 µs**

